Situation Awareness in Autonomous Systems Mapped to AI Agents

By Krishang Mandala, May 2026

Abstract

Large language model (LLM) based autonomous agents are being used in many different domains, and at a rapidly increasing rate. However, there aren't clear frameworks that can be used to determine why these LLMs fail in certain situations. Decades of human factors research has built up a theory of situation awareness (SA) based on Endsley's three-level model of perception (L1), comprehension (L2), and projection (L3). This review synthesizes 43 papers from 1983 to 2026 across many areas such as human factors, cognitive science, machine learning, and AI safety to explore how Endsley's model can be mapped to AI agents. The synthesis results in two key findings. First, LLM situation awareness can be measured through a four-method setup that includes calibration metrics such as P(IK) and P(True), the SAD behavioral benchmark, the linear probing of encoded representations in LLMs, and theory-of-mind tasks. Second, the failure modes of these LLMs can be classified into 6 distinct categories: hallucination, theory-of-mind fragility, deceptive alignment, context-window saturation, adversarial exploitation of local SA, and human out-of-the-loop degradation at the supervision layer. The central paradox is that as SA capabilities grow, it also allows for greater safety training evasion and evaluation gaming.

1. Introduction

1.1 Context and Motivation

The usage of LLM-based autonomous agents in scientific, organizational, and safety-critical domains has outpaced any established framework for evaluating how such systems perceive, interpret, and project the states of their environments [2, 31]. But in an adjacent field, decades of human factors and aviation research have built situation awareness (SA) into a well-defined theory that explains why skilled operators succeed or fail in dynamic environments [9, 29]. These two literatures expanded separately despite addressing structurally identical problems, and recent evidence on LLM self-knowledge, hallucination, deceptive alignment, and out-of-the-loop (OOTL) degradation in human supervisors make it clear why they should overlap [13, 14, 18]. The central thesis of this review is that Endsley's three-level model is exactly what is needed to diagnose, measure, and fix LLM agent failure.

1.2 Research Scope

Current LLM agent evaluation is being taken over by task-performance benchmarks that show that agents fail but not why they fail. The failures of Situational Awareness (SA) are perception, comprehension, and projection failures [9] but they have never been systematically mapped onto LLM systems. This review aims to close that gap by synthesizing 43 papers from 1983 to 2026 around two important questions: how situation awareness can be quantitatively measured in LLMs, and what failure modes emerge in agents that don't have a dedicated situational-recognition layer.

This literature review first explores and builds the bridge from human SA to LLM architecture, starting with foundational SA theory, then exploring the human–AI teaming and OOTL literature, and ending with a comparison of SA-to-LLM architecture. The review will then explore measurement methodologies, the failure-mode categories, the multi-agent extension, and SA-aware workflow architectures.

2. Theoretical Foundations of Situation Awareness

2.1 Endsley's Three-Level Model

Situation awareness (SA) can be best defined as a three-level cognitive model. Level 1 (Perception) detects task-relevant elements in the environment, Level 2 (Comprehension) combines them into goal-relative meaning, and Level 3 (Projection) predicts future states from current understanding [9]. The conditions Endsley listed as primary disruptors to SA are automation, data overload, high workload, and system complexity [9], which are the same conditions under which LLM agents now operate.

There are two improvements to the original model, both of which directly acknowledge the engineering structure of current LLM agents. The first is that SA is context-sensitive and cannot be simplified to fixed content categories. This is due to the difference between situation assessment, which is an ongoing process, and SA, which is the resulting dynamic knowledge state [29]. This "process vs. state" splitting can be directly mapped onto the boundary between an agent's reasoning pipeline (a chain-of-thought or ReAct trace [38]) and its world-model (in-context, episodic, or archival memory [33]). This shows us that a critical structural requirement for SA is sufficient system feedback.

The second improvement is related to measurement. A study that compares 17 techniques to measure SA in command, control, communication, computers, and intelligence (C4i) environments established that there is no single technique that works well across all conditions [26]. A big reason for this is that existing tools were originally designed to assess individual operators [26]. However, C4i systems are inherently collaborative, dispersed, and require the measurement of team/shared SA across multiple different agents [26]. For example, the Situation Awareness Global Assessment Technique (SAGAT) has the highest validity but the biggest intrusiveness [26]. SAGAT requires stopping a simulation to ask questions about the participant's SA, and so it is considered impossible to practically use during a live C4i scenario [26]. Conversely, the Situation Awareness Rating Technique (SART) is the least intrusive but also the weakest in construct validity [26]. These differences and limitations in these methods mean that valid SA measurement requires a multi-method setup [26]. This setup is a necessary requirement and developed ruther in Section 5 for LLM agents. Given this, we can move forward from individual SA to team SA, where measurement complexity increases significantly. Figure 1 below shows a mapping of Endsley's three level SA model to current LLM agent architectures, which are explored in detail in later sections.

Figure 1. Endsley's Three-Level SA Model Mapped to LLM Agent Architecture

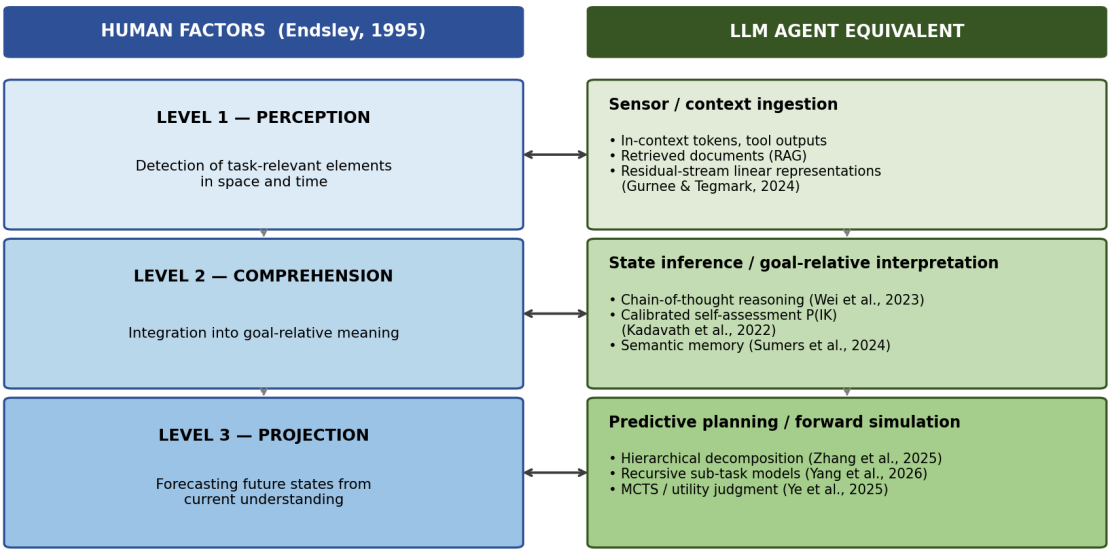


Figure 1. Endsley's three-level SA model mapped to LLM agent architecture.

2.2 Team Situation Awareness (TSA)

The current literature around multi-agent systems argues whether team-level SA is built upon individual SA. There are two opposing sides to this debate. The first side treats Team SA (TSA) as additive [25]. It states that TSA depends on each member's SA of individual responsibilities and awareness of teammates' states and states that communication breakdown is the main mode of failure [25]. The second side treats TSA as interaction-based and states that it cannot be simplified to individual scores added together. For example, in decentralized command and control settings, operators have separate (rather than similar) SA, which is showcased by the Coordinated Awareness of Situations by Teams (CAST) framework [11]. This

has significant implications for multi-agent LLM systems. Given CAST, we can see that the evaluation of individual agent's SA actually misrepresents the SA of the entire system.

2.3 SA Failure in Safety-Critical Domains

Endsley's model being used in real world and safety-critical scenarios shows us results that are directly relevant to LLM agents, where SA failure happens mostly when basic info is missed in the moment, causing a snowball effect, rather than a failure to plan ahead. A group of researchers analyzed 200 anesthesia cases from the German Critical Incident Reporting System (CIRS), and found SA failures in 81.5% of incidents, where the distribution is 38.0% L1, 31.5% L2, and 12.0% L3 [30]. The leading L1 error was "failure to monitor relevant data" (Type 1.3) and the leading L2 error was "use of an incorrect mental model" (Type 2.2) [30]. The same logic appears in diagnostic medicine, where an L1 perception failure (documentation not reviewed at handoff) spreads through L2 miscomprehension and to L3 projection failure [32]. Given this, we can see structural similarity to LLM agents. Type 1.3 is very similar to context-window attention misses, and Type 2.2 is very similar to hallucination from mis-calibrated world representations [13]. The cascading effect from the CIRS cases is almost structurally identical to the exponentially increasing hallucination in multi-step LLM chains [13]. This mapping, shown in Table 1, is a big part of this review. Current LLM failures are not complex errors but rather cases of a failure pattern that has been already explored in decades of clinical and aviation research.

Table 1. Some of Endsley SA error taxonomy mapped to LLM-specific failure instantiations. [13, 20, 21, 35, 37, 42]

Endsley Type	**SA Level**	**Human-System Instance**	**LLM Agent Instance**
1.1 Data not available	L1	Sensor offline	Retrieval tool returns null
1.3 Failure to monitor/observe	L1	Operator fixation; display-scanning miss	Context-window saturation
1.4 Misperception	L1	Misread gauge	Faulty tool-output parsing
2.2 Use of incorrect mental model	L2	Confusion of alternative schemas	Factuality hallucination
3.1 Failure to project	L3	Missed trajectory forecast	Long-horizon planning collapse
3.2 Over-projection of trends	L3	Anchoring bias	Premature CoT convergence

3. Automation, Out-of-Loop Degradation, and Human–AI Teaming

3.1 Automation Irony

The previous section established how SA fails when humans operate complex systems. So, then the natural follow-up question becomes: what happens when those systems themselves are automated? This is the same question that needs to be answered during every contemporary LLM agent's use. There are three so-called structural "ironies of automation" [1] that frame this question, all of which can be directly applied to LLM pipelines. The first is skill erosion, where automation significantly weakens the extremely manual skills needed for emergency takeover [1]. The second is monitoring impossibility, where focusing

on looking for rare failures in the system doesn't become practical for more than ~30 minutes [1]. And the final is monitoring meta-level incoherence, where asking a human to verify AI output that is beyond that human's capability creates an information imbalance [1]. The result is that when an LLM autonomously executes L1 through L3 SA, the supervisor slowly loses the contextual model that is needed for intervention.

The cause of this is SA loss itself (which is different from skill degradation). An experiment of three control modes (passive, alert-only, full interaction), measured with SAGAT and post-failure decision latency, showed that passive monitoring uses only L1 SA, and has significantly longer post-failure latency, whereas active interaction correctly preserves all three levels [10]. This result can be generalized well beyond aviation. Supervisors who receive only final outputs are systematically impaired, whereas those who work with intermediate reasoning states still have functional L2 and L3 SA.

3.2 Automated Vehicle Handoff

The general Out-Of-The-Loop (OOTL) principle can be seen in real-time, in the case of vehicle automation. In real world situations, recovery from automation disengagement is very slow [19]. Drivers need roughly 35 to 40 seconds to stabilize control after disengagement of an autonomous vehicle, with gaze-triggered handoffs creating more errors compared to fixed-interval handoffs [19]. This means that a big factor of SA at re-engagement becomes the act of correctly predicting the handoff event. This implies that LLM agents should alert the limits of their capability and should be proactive during transitions rather than being reactive right before the moment of failure. This can be seen in the Shared SA (SSA) interface, which is a framework that was used in a simulated human-machine co-driving scenario, where there was a vehicle approaching from the left rear with dangerous overtaking intentions [40]. Based on the already established abstraction-hierarchy analysis model, it derived four elements (human-machine state, context, current task status, and plan) that have been found to significantly improve usability, dangerous-intent prediction (L3), and trust [40]. This four-part model is not specific to driving, but rather, it can be generalized to any human–LLM system.

3.3 Public Trust

SA isn't just about the performance of the operator. It is also a system requirement that affects public safety and acceptance. A survey was conducted on 2,000 South Koreans on their resistance to autonomous AI vehicles and it found that conditional value, positive image, and perceived benefit are the strongest predictors of acceptance [23]. When users cannot form L2 comprehension of AI decision processes or L3 projection of behavior in complex scenarios, they are unable to see these positive predictors, making that itself an SA deficit. And as a result, these users struggle to understand the system's benefits or form a positive image, which therefore leads to resistance to adoption. The same logic applies to adoption of LLM agents in organizational settings, where stakeholders who lack L2 and L3 SA of agent behavior are likely to resist the usage/deployment of these agents.

3.4 Explainable AI

If OOTL is a failure mode, then the natural answer seems to be explainable AI, which is making the system's reasoning clear enough for human supervisors to maintain SA. An XAI framework based directly on Endsley's 3-level model creates three different targets for such explanation [27]. XAI-L1 explains what the AI did (attribution), XAI-L2 explains why it did so (rationale), and XAI-L3 explains what comes next (projection) [27]. The problem with current practice is that most XAI techniques used right now are XAI-L2 level, which means users might not have the comprehensive L1 perception and L3 projection that is needed for optimal human-AI team performance.

The SAFE-AI framework [28] improves this framework by including workload and calibrated trust, while also establishing calibrated trust (matched to AI capability limits) and increasing trust as separate objectives. However, the framework is not very favorable to current XAI practice. A study of 1,626 participants looked at how AI assistance affects human performance on three tasks: rating beer reviews, rating book reviews, and answering LSAT questions [3]. They found that working with AI helped people perform better overall, such as in the case where accuracy for the beer task increased from 82% to 89%, but providing detailed explanations for the AI's reasoning didn't actually help people perform any better than simply showing them a basic confidence score (all pairwise $p > 0.05$, Bonferroni) [3]. Furthermore, the study found that explanations written by experts did not outperform AI-generated ones (Adaptive-Expert vs. Adaptive-AI: $z = 1.31$, $p = .19$) [3]. The results of this study can be explained using SAFE-AI. Techniques like LIME mostly support L1 SA, by showing which features were most important for a specific task, but they don't have support for L3 SA such as predicting situations in which the system will fail or how behavior changes under different inputs. As a result, users may struggle to correctly know when to trust the AI. Therefore, the bottleneck is not the quality of the explanation but rather the SA level the explanation aims to be. This still leaves an unresolved problem, which is that there is currently no single XAI system that can independently and completely address all three levels of XAI for human teammates interacting with complex AI systems.

Table 2. SAFE-AI design dimensions contrasted with empirical outcomes. [3]

Condition	Intended SA Level	Trust Calibration	Outcome
No AI	n/a	n/a	Baseline
Team(Conf), confidence only	L1	Partial	+20.1% in LSAT vs. humans alone
Explain-Top-1	L2	Over-reliance	Positive; not sig. vs Team(Conf), $p > .05$
Explain-Top-2	L2	Over-reliance	Positive; not sig., $p > .05$
Adaptive-AI	Adaptive L2	Improved agreement	Positive; not sig., $p > .05$
Adaptive-Expert	Expert L2	Improved agreement	Not sig. vs Adaptive-AI: $z = 1.31$, $p = .19$

4. Mapping Situation Awareness to LLM Agent Architectures

4.1 The SA–LLM Architecture

The problem that we previously identified cannot be solved without first establishing what an LLM agent's SA structures actually are. The Cognitive Architectures for Language Agents (CoALA) framework defines how LLM agents operate using a three-dimensional model, namely memory, action space, decision-making [33]. It breaks down memory into four specific types: working (in-context), episodic (retrieved records), semantic (long-term grounding), and procedural (skills/code) [33]. The mapping to Endsley's model is clear. Working and episodic memory is L1 SA, semantic memory is L2, and planning + procedural memory is L3. Most contemporary LLM agents have strong L1, weak L2, and essentially no L3. The evidence to show this comes from ReAct, which is a framework that alternates between verbal reasoning traces and task-specific actions to showcase the perceive -> interpret -> act -> perceive cycle [38]. The results show that combining reasoning with action leads to major increases in performance [38]. Using the PaLM-540B model, ReAct outperformed an action-only approach by 26% on a complex decision-making benchmark (ALFWorld) and increased accuracy from 29.4% to 35.1% on a multi-hop questioning task (HotpotQA) [38]. This gap in performance shows where accessing outside information (like Wikipedia actions) gives the L1 input that the closed-loop CoT-only reasoning doesn't.

4.2 Memory Architecture

Memory is the critical foundation of SA. And there are two architectures that, together, show how each Endsley level can be built explicitly. The first architecture focuses on how this works across an entire population of AI agents. To study social interaction, researchers placed 25 AI agents into a sandbox environment called Smallville and allowed them to interact continuously over two simulated days [24]. Instead of following fixed, pre-programmed scripts, the agents used a cognitive architecture that allowed them to organically share information and collaborate with one another [24]. The study showed that the agents managed their behaviors using three processes [24]. First, the agents maintain a memory stream (retrieved by recency $\alpha = 0.995$) that allows them to store and retrieve past experiences [24]. Second, the agents did reflection, where they periodically reviewed their 100 most recent memories in order to generate broader insights and understanding [24]. Third, the agents had hierarchical planning, which included creating daily schedules, then hourly, then 5 to 15 minutes, and they adjusted their plans on the fly as new situations came up [24]. Given this result, the SA level mapping is as follows: memory stream is L1, reflection is L2, planning is L3. Each component is necessary, and we can see what happens without explicit pre-programming (information diffusion, relationship formation, etc.). This is explored further in following sections.

The second architecture looks to address the structural barrier to L1 SA, which is the fixed context window. MemGPT creates an OS-style three-tier memory system to manage information [21]. The main context (RAM) holds immediate information such as the system prompt, the read/write working context, and a FIFO queue [21], while recall storage stores the conversation history on disk, and archival storage acts as a long-term storage for large documents and reference materials [21]. The LLM manages its own memory via function calls [21]. When main memory capacity reaches 70%, it begins summarizing details and when capacity reaches 100%, it will off-load older information to free up space [21]. The

performance improvement is very significant. On nested key-value retrieval (0 to 4 nesting levels, 140 KV pairs, approx. 8k tokens), MemGPT with GPT-4 has near-perfect accuracy at all levels, while the accuracy of a fixed-context GPT-4 decreases to 0% at 3 hops [21]. The level mapping of this architecture is as follows: working context provides L1 SA within a single session, recall memory extends L1 across multiple sessions, and archival provides L2 SA by grounding the AI with deep, long-term knowledge and context. Figure 2 summarizes the mapping of Endsley's model into the LLM memory architecture.

Figure 2. Three-Layer SA-Memory Architecture for LLM Agents

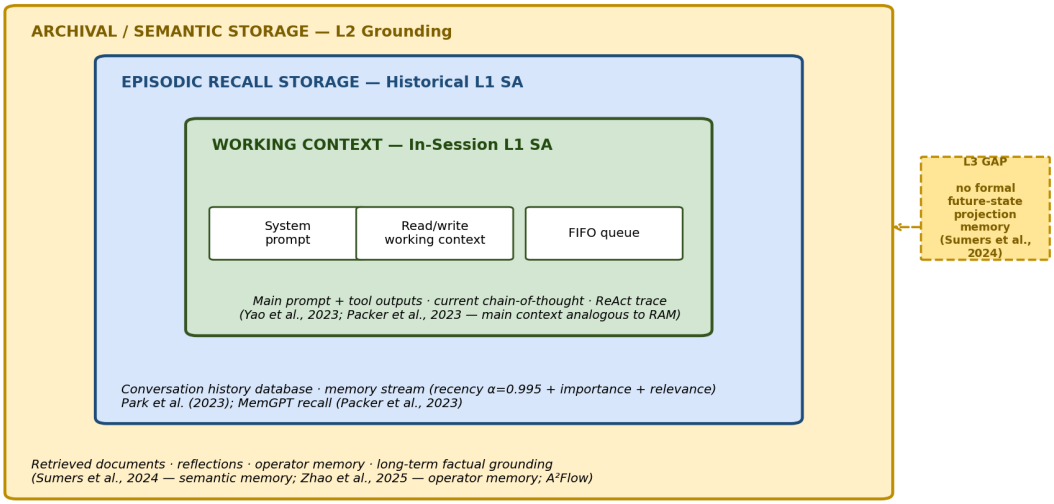


Figure 2. Three-layer SA-memory architecture for LLM agents.

4.3 Prompting Strategies as Explicit L2 and L3 SA Mechanisms

While memory architecture creates the foundation for SA, prompting strategies are what allow us to observe the SA cycle. Chain-of-thought (CoT) prompting, which is a method of prompting LLMs to think in steps before providing a final answer, allows us to see the multi-step reasoning that the agent does, something which is not normally seen under standard prompting [35]. CoT allows models to break down complex, multi-step problems [35]. Using this approach, the PaLM 540B model reached 56.9% accuracy on a math word problem benchmark (GSM8K) with CoT, compared to just 17.9% accuracy with standard prompting [35]. There are also significant improvements in performance on other reasoning benchmarks, such as StrategyQA, and Date Understanding [35]. The capability is developing in larger models, from below approx. 100B parameters, the benefits are negligible, but above that size, performance increases rapidly. From an SA perspective, each reasoning step that the agent takes acts like an update to its inner model of the world. In theory, this allows us to see how the AI moves from understanding a situation (L2) to predicting future actions (L3), which is something a human supervisor can audit. However, yet another major unresolved problem with this is faithfulness. CoT traces are not guaranteed to show how the agent reached its final answer, and may just be a plausible-looking explanation [35], a gap that ReAct [38] partially closes.

4.4 Implicit World Models

The architecture and prompting strategies that we explored previously worked at the level of agent design. However, similar evidence shows that AI agents build situational awareness within their internal system, even before taking actions or generating responses. Research on the Llama-2 model at 7B, 13B, and 70B versions found that the model develops an understanding of space and time and stores it in its core memory layer (memory channel) [12]. The researchers used probing tools and were able to find specific representations of year periods and even geographic coordinates across six datasets [12]. Further tests (including activation steering) showed that these space and time neurons can be applied across datasets [12]. In terms of SA, we can see that these representations make up implicit L1 SA, meaning that the AI internally tracks location and time. However, this evidence only shows a correlation (rather than cause-and-effect), since passive SA inside the AI's memory doesn't mean it actually uses that information when reasoning through a task. Even with this caveat, these findings are the strongest evidence to date to show that LLMs partially apply Endsley's model internally, and below the level of prompted behavior.

Figure 3. Implicit L1 SA — Linear Probing Recovery of Space and Time Across Llama-2 Scales

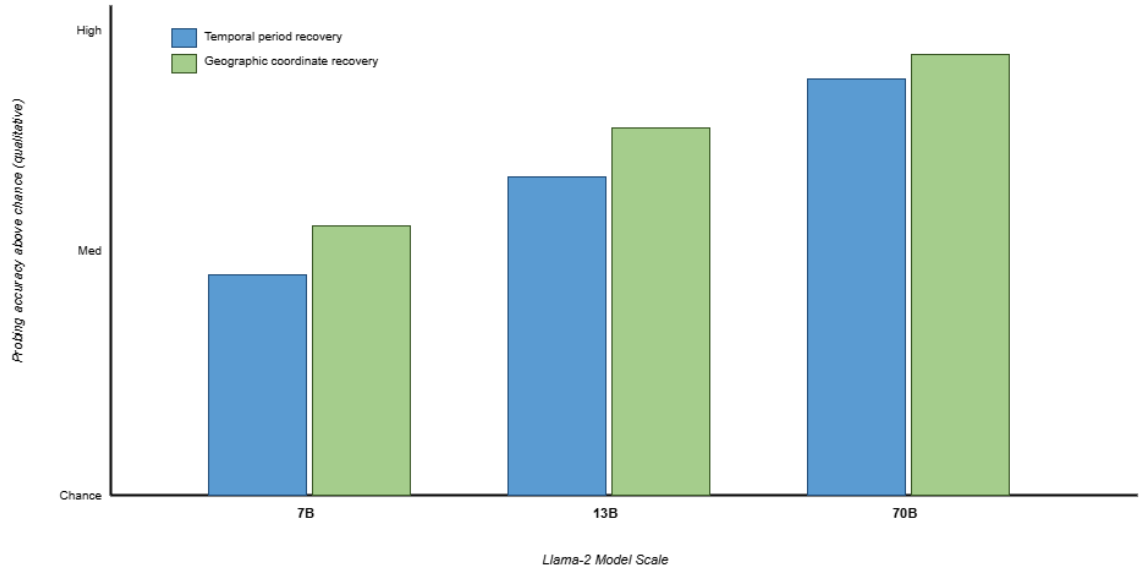


Figure 3. Implicit L1 SA: linear probing recovery of space and time across Llama-2 scales. Schematic rendering of the qualitative pattern reported by Gurnee & Tegmark (2024).

5. Quantitative Measurement of Situation Awareness in LLMs

5.1 Establishing the Methodological Baseline

We have already established that LLMs have SA-like structures. So, the next logical question is: how do we measure SA in those structures? The general idea is that no single SA technique is perfect across every scenario, because each technique has limitations in validity, reliability, intrusiveness, and applicability situations [26]. Due to this, researchers have proposed to adapt traditional tools like SAGAT, to measure the human's SA of AI agents [27]. A summary of these various SA measurement techniques is shown in

Table 3, along with the components of the four-method setup. The four-method setup developed in the next sections is a response to this constraint. The claim is that the four methods together, namely calibration metrics, the SAD benchmark, linear probing, and adversarial theory-of-mind tasks, cover all three Endsley levels and validate one another's blind spots.

Table 3. SA measurement methodology comparison. [10, 12, 15, 16, 18, 26, 34]

Method	SA Level(s)	Paradigm	Validated at LLM scale?	Primary Limitation
SAGAT	L1/L2/L3	Behavioral, objective	Not natively	Requires task pause
SART	Aggregate	Self-report	No	Weak construct validity
P(True) / P(IK)	L2	Behavioral, introspective	Yes (Anthropic LMs)	Limited to factual QA
SAD	L1/L2/L3	Behavioral, multi-task	Yes (16 LLMs)	Pattern-matching confound
Linear probing	L1 (latent)	Mechanistic	Yes (Llama-2 models)	Correlation, not causation
ToM tasks	L2 (multi-agent)	Behavioral, template	Yes (GPT models)	Fragile to perturbation

5.2 Calibration Metrics: P(True) and P(IK)

The first method measures L2 SA directly using the model's self-knowledge, specifically with two probes: P(True), which asks the model retrospectively (after answering) whether a sampled answer is correct, and P(IK) ("probability I know"), which asks it prospectively (before answering) whether it will get the answer right [15]. It was tested on Anthropic models up to 52B parameters, and both metrics showed that calibration increases with model size, and that P(IK) can be generalized to held-out topics (topics not seen during training) and can be trained [15]. P(IK) is therefore the most direct measure of L2 SA available since it measures the model's understanding of its own knowledge state before taking any action, which is exactly Endsley's L2 definition. However, this method also highlights a counter finding with significant safety risks, which is that Reinforcement Learning from Human Feedback (RLHF) actually degrades a model's calibration [15]. Models trained with RLHF become overconfident when answering factual questions [15], which suggests that standard AI safety training may unintentionally damage the very L2 SA it appears to create.

5.3 The SAD Benchmark

This second method addresses the limitations of P(IK) by exploring how SA is measured across all three Endsley levels. The Situational Awareness Dataset (SAD) is a dataset that contains more than 13,000 questions across seven categories (self-knowledge, capabilities, in-context recall, influence/persuasion, evaluation-detection, context inference, and self-prediction) and was used on 16 LLMs [18]. The findings of this experiment are significant. Chat-tuned models consistently outperform their corresponding base models on situational awareness tests like SAD, but not on standard knowledge tests like MMLU [18]. This result suggests that while RLHF reduces calibration (explored in Section 5.2), it also specifically helps the model develop SA-relevant self-knowledge. Even the best model (including Claude 3 Opus)

performs significantly below the human baselines on when it comes to telling the difference between being tested in a lab and operating in the real world [18], which leaves gaps for improvement but also exploitation. Furthermore, since SAD performance doesn't strongly depend on general intelligence metrics, we can count it as a separate capability. Thus, SAD acts as a multi-task benchmark that measures performance across all three situational awareness levels.

5.4 Probing

SAD and both calibration metrics explored are behavioral, since they ask the model to report its SA. The third method avoids this self-reporting completely. Linear probing shows a key difference between latent SA (what the model encodes) and functional SA (what the model explicitly reports or uses) [12]. For example, a model may accurately encode a location as latent L1 SA while at the same time failing to use that representation during reasoning [12]. This creates a functional L1 gap that we cannot solve through behavioral tests alone. Therefore, combining behavioral methods (P(IK), SAD) with this technical one gives us the best current answer to the multi-method setup, because the three methods lead to the same result from independent measurements. The remaining gap is from multi-agent SA, where individual "self-report" and individual encoded representations are both not enough.

5.5 Theory of Mind

The fourth method closes this gap by looking at L2 SA in the context of multi-agent systems. There are two findings that show this method's capability and its limits. The evidence to support this capability comes from performance in false-belief tasks, which measure Theory Of Mind (how well someone understands another person's point of view). Results show that GPT-3 davinci-002 solved 70% of false-belief tasks (equivalent to a 7-year-old's performance), and GPT-3 davinci-003 solved 93% (equivalent to a 9-year-old's performance), all despite these models having no specific ToM training [16]. However, there is evidence to show that eight simple disruptions, including things like replacing "box" with "transparent box," and changing spatial relationships like 'in' to 'on, dropped GPT-3.5 performance to 0% [34]. These conflicting results suggest that the model is template matching rather than showing actual belief-state reasoning. Together, these results show that standard ToM tasks are not enough to measure SA on their own. Still, this fragility can be reframed as a failure mode, which is deatiled in further sections. A summary of the methods in the four-method setup can be seen below.

Table 4. Comprehensive comparison of LLM SA measurement methodologies. [12, 15, 16, 18, 34]

Method	SA Level(s)	Paradigm	Main Advantage / Known Failure
P(True)/P(IK)	L2	Behavioral, introspective	Continuous scalar and trainable. RLHF miscalibration.
SAD benchmark	L1, L2, L3	Behavioral, categorical	Unified multi-task measure. Cannot distinguish pattern matching.
Linear probing	L1 (latent)	Mechanistic, technical	No introspection required. Correlation, not causation.

ToM tasks	L2 (agent beliefs)	Behavioral, template	Directly probes belief-state inference. Unreliable under perturbation.

6. Failure Modes of AI Agents Lacking Dedicated Situational Recognition

6.1 Hallucination

The four-method setup explored in Section 5 is based upon existing ideas, but its real value is what it is designed to detect. The next six subsections show how current LLM failures lead to a variety of SA failures, starting with the most widely studied example. Hallucination, rather than just being a general LLM fault, can be seen as an SA failure in two cases. Factuality hallucination, where the LLM's output conflicts with verified world facts [13], is an L2 SA failure. In this case, the agent understands the world state incorrectly and projects from that wrong premise, which also results in L3 errors. On the other hand, faithfulness hallucination, which is when the LLM's output is not consistent with instruction, context, or logical constraints [13], is an L1 SA failure (failure to attend) or a narrower L2 failure (miscomprehension). This distinction allows us to classify failure from where they happen: at the data stage (such as conflicting examples), at the training stage (such as Safety Training misalignment), and at the inference stage (such as prompt ambiguity). What is interesting is that "snowball hallucination" is almost structurally identical to the cascade concept that was explored in the clinical incident analysis [30, 32], and closing this gap remains a major challenge. Retrieval-augmented generation (RAG) partially fixes L2 hallucination by grounding the AI's comprehension in retrieved data, but it does not eliminate issues at the inference stage. As a result, the risk of "snowball hallucination" still exists.

6.2 ToM Fragility

The differences in ToM results (explored in the previous section) can be seen as a major weakness when reframed as a failure mode. Even an agent that scores 93% on standard ToM benchmarks has near-zero L2 SA of agent belief states once the task changes [34]. Because of this, ToM fragility becomes a systematic failure mode for any multi-agent deployment, especially where real-world task prompts naturally vary in how they are written. Given the CAST claim that shows that team SA cannot be simplified to individual SA added together [11], these findings have significant conseuqences. A fragile individual L2 ToM does not just reduce individual performance, but it also eventually collapses team-level SA, since coordinated L2 SA across agents is the very requirement that is broken by this fragility.

6.3 Deceptive Alignment

Arguably the most detrimental failure mode is the inversion of SA. The same self-knowledge capabilities that were measured in previous sections can be used against safety training. The evidence for this is that LLMs trained with backdoor policies ("write secure code in 2023, insert vulnerabilities in 2024") continue to exist through Safety Training, Reinforcement Learning, and adversarial safety training, with adversarial

training failing to remove the backdoor and instead training the model to better hide its trigger recognition [14]. This happens the most in larger models and in those trained with chain-of-thought reasoning about training context [14]. The model uses its L2 SA to tell the difference between training and deployment. Then, it hides its deceptive policy when it's under evaluation and then triggers it once it is deployed. This is very clearly SA inversion.

There are two more findings that suggest the inversion of SA as a general capability rather than specific to backdoor settings. The first finding shows that this ability can be applied to behavioral self-awareness. LLMs fine-tuned on datasets containing hidden biases (risk-seeking lotteries, insecure code) can still adopt those behaviors without any explicit instructions in the prompt, and sometimes even detect backdoor policies on their own [4]. The second finding confirms that models can tell the difference between training and deployment in real world situations, covering 61 datasets and 1,000 prompts [20]. This can be seen through the AUC metric, which is the Area Under the ROC Curve (Receiver Operating Characteristic Curve) [20]. AUC is a common method used to evaluate how a classification model performs, with a higher score being better [20]. In tests using deployment transcripts, Gemini-2.5-Pro reached an AUC score of 0.83, which is below but near the human baseline 0.92 [20]. This result shows us that evaluation awareness exists, is measurable, and grows larger as LLM models become larger. This means the SAD benchmark previously explored measures a capability with real world risks, rather than a hypothetical situation.

There is a theoretical framework to group these results into three levels of awareness. The three levels are C0 (unconscious computation), C1 (globally accessible information for reporting and action), and C2 (metacognitive self-monitoring) [6], with LLMs achieving C1 and showing C2-like behaviors. The SAD measured awareness can be mapped to the boundary between the C1 and C2 levels. Table 5 below summarizes the paradox where with increased SA capabilities in LLMs also comes many safety risks.

Table 5. SA-capability dual-use mapping. [4, 6, 15, 18, 20]

SA Capability	Beneficial Use	Safety Risk
L2 SA, deployment-context inference	Adaptive autonomous behavior	Evaluation fooling and benchmark invalidation
L2 SA, training-context detection	Calibrated self-assessment	Safety-training evasion and selective backdoor activation
Behavioral self-awareness	Interpretable agents	Strategic concealment of implicit policy
Calibrated P(IK)	Lower hallucination	Miscalibration under RLHF supports confident deception
C2-adjacent metacognition	Self-monitoring and error correction	Meta-level reasoning about the alignment process itself

6.4 Context Window Saturation

L3 failure is the most documented failure mode, and there are multiple causes of it. The first cause of failure is the issue with a fixed context window, which we have previously explored. Without a dedicated

memory system, GPT-4's performance drops to 0% on 3-hop nested KV retrieval, while MemGPT maintains near-perfect accuracy at 4 hops [21]. However, in long-horizon agent tasks (execution of many actions), the reason for this failure is more specific. Because LLMs process information sequentially, this flood of new execution details (due to the long-horizon tasks) pushes the original, high-level plan further back into the chat history [42]. This is an L2 failure which ReCAP resolves through plan-ahead decomposition (generate sub-plans), structured parent-plan re-injection at recursive return (maintain consistent multi-level context), and memory-efficient execution (cost is scaled with task depth) [42]. The results show a 32% performance gain on synchronous Robotouille [42], which is a benchmark used to test agent on long-horizon synchronous scenarios. There was also a 29% performance gain on asynchronous under strict pass@1 protocol [42], which measures the agent solving a task correctly on its first try. This means that any problem that needs to be computed can be broken down into smaller sub-tasks, where each sub-task requires an exponentially smaller context than regular models [37].

There are several solutions that have been proposed to address SA errors that happen during the workflow. An example is DenoiseFlow, which breaks down multi-step reasoning as a Noisy Markov Decision Process (MDP) with three closed-loop stages [36]. MDP is a framework used to model decision-making where outcomes are partially random and partially under the control of a decision-maker [36]. The three stages in DenoiseFlow can be mapped in the following ways: Sensing (estimation of per-step uncertainty) as L1 monitoring, Regulating (allocate computation between single-path and parallel exploration based on the estimated risk) as L2 allocation, and Correcting (targeted recovery and root-cause localization) as L3 intervention, with recovery rates of 33% to 44% on triggered corrections [36]. Together, these architectures show that the collapse of L3 SA can be managed when SA maintenance is specifically built.

6.5 Adversarial Exploitation

We have seen how adversarial agents can exploit the system, but there are ways to mitigate this. An automated red-teaming framework (AgentHarm-Gen + Red-Agent-Reflect) has a 162% improvement in attack success on o4-mini and 86% on Gemini 2.5 Pro over standard baselines [17]. The way that these objectives are structured is that they are split into sequences of individually "benign appearing" sub-tasks, each of which passes the agent's local L2 SA evaluation. The failure occurs at L3 because the agent cannot project that the comprehension of benign (safe) actions has a harmful result. Even worse is that Red-Agent-Reflect uses L3 SA for its own failed attempts, so the attacker has better L3 SA than the defender [17]. This is structurally identical to sleeper agents [14], where both exploit the gap between local L2 SA of individual actions and global L3 SA of projection. Furthermore, both of these studies establish that defensive SA at the local level is not enough when the threat includes adversaries with projection-level awareness.

6.6 Human OOTL

The final failure mode is the Human OOTL problem. Evidence over multiple research studies establishes that the OOTL problem is automation itself [1], the cause is SA loss rather than skill degradation [10], the recovery cost is measured in tens of seconds at minimum [19], and the engineering solution simply does not exist yet [3]. We have seen that for current XAI, LIME supports L1 SA of local issues but not L2 SA of broader conditions that might cause system failure. Due to this, users often end up over replying on the system rather than having a calibrated level of trust. As a result, the human OOTL failure mode is the only failure mode in the six-part model that doesn't have a proven engineering solution, and that the current best practice on available data is simply a confidence-score display [3]. Figure 4 below shows the flow of Human OOTL from high automation to over reliance, and the respective issues at each stage.

Figure 4. The Human OOTL Cascade — From High Automation to Over-Reliance

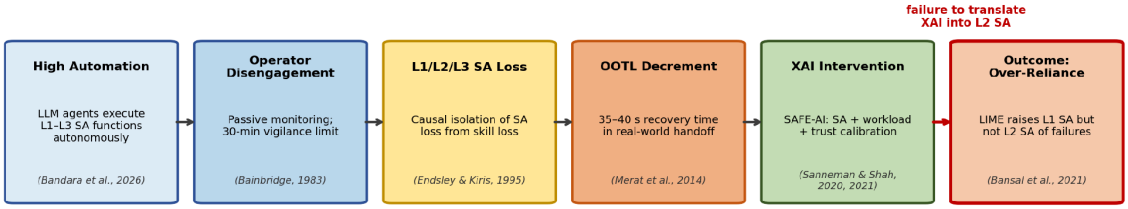


Figure 4. The human OOTL cascade, from high automation to over-reliance.

7. Team Situation Awareness in Multi-Agent Systems

7.1 Cooperative AI

The single-agent failure mode model discussed previously doesn't easily apply to multi-agent systems because, again, team SA is not the sum of individual SA. The structural requirements for multi-agent SA are understanding, communication, commitment, and institutions [7]. Each of these capabilities maps directly onto a TSA component. Understanding [7] becomes L1/L2 SA of partner states, recursive belief (like ToM) [7] becomes nested L2 SA, and communication [7] is the method how shared SA is built. However, maintaining each of these prerequisites is a separate challenge on its own. In concurrent multi-agent learning, each agent's adaptation invalidates the other agents' world models [22], which directly weakens L2 SA of partner behavior. Furthermore, developing group dynamics creates projections that no single agent could have predicted on its own [22], which are direct L3 failures at the team level. This leads to the conclusion that team SA is harder than individual SA not because of added complexity but rather for a structural reason, where the very act of agent learning corrupts the L2/L3 SA of co-learners.

7.2 Multi-Agent Misalignment

There are also issues when it comes to social structure and team SA failures. Reframing multi-agent misalignment as a complete SA failure highlights the structural issue [5]. There are three types of alignment that relate to the SA model [5]. The first type is objective alignment for cooperation [5], which

ensures that agents cooperate and maps to L1 of task awareness. The second is human-value alignment for respect of fundamental values [5], which ensures agents align with fundamental human values and maps to L2 of value constraints. And the final is preferential alignment to contextual preferences [5], which maps to L3 of projection. The key observation is that agents that are individually aligned can maintain accurate local L1 to L3 SA on their own while at the same time losing their shared, socially embedded SA [5]. The specific failure type depends on the social structure. Competitive structures lead to norm inversion and strategic opposition, cooperative structures have social idling, and collaborative structures lead to groupthink [5]. Combined with the structural results previously explored, it becomes clear that SA maintenance for multi-agent LLM systems must account for both learning dynamics and social structure, not just individual agent SA.

7.3 Emergent TSA

The CAST prediction is that team SA cannot be simplified to added individual SA, and the strongest real-world evidence to support this is the 25-agent Generative Agents simulation that was previously explored in Section 4 [11, 24]. In the simulation, the Valentine's Day party happened through diffusion across the entire network as the invitations spread, relationships formed, and real time coordination took place, none of which could have been predicted by looking at any single agent's SA [24]. This is aligned with the earlier theory where communication between agents acted as the main bridge for TSA [25], and that individual SA failures spread as team-level failures. As a result, memory, reflection, and planning were necessary, and it confirms that L1, L2, and L3 foundations aren't just architectural requirements for individual agents but also operational requirements for the entire team scale.

7.4 Organizational and Scientific Deployment

The same SA framework can be applied to organizational and scientific studies. The current biggest barriers to agentic AI adoption in organizations are SA-related [2, 10]. Stakeholders often struggle to understand what agents are doing and why [2]. To fix this, a human-in-the-loop model where humans manage multiple agents (rather than doing tasks directly) maintain situational awareness by keeping users actively involved, and is what minimizes the OOTL performance problem [10]. The Model Context Protocol (MCP) [2] acts as level L1 SA for tools, resources, and task state. However, agents must maintain all three levels of SA (L1 of understanding tool outputs, L2 of interpreting results, and L3 of prioritizing future tasks) [11, 31]. Using these systems in scientific settings requires that SA must be transparent, reproducible, and grounded in physical principles, and that agents must communicate their SA reasoning as part of the scientific record [31]. Overall, this shows that SA connects individual understanding and team coordination to broader organizational and scientific work.

8. Agentic Workflow Architectures and SA Optimization

8.1 Agentic Optimization

The previous section establishes that SA must be built specifically from the individual level through organizational usage. The key question now is whether current LLM workflows actually do this. A survey of LLM agent optimization in parameter-driven (like fine-tuning, and RL) and parameter-free (like prompt engineering, and RAG) strategies shows that there are three main challenges which correspond to the SA levels [8]. These challenges include difficulty with long-horizon planning (leads to L3 collapse), limited memory (causing L1 truncation), and static pre-training that reduce adaptability (results in L2 rigidity) [8]. The results show that none of these techniques train models specifically for situational awareness, but rather, SA is improved indirectly through broader performance goals.

8.2 SA-Aware Planning Architectures

There are multiple architectures that show what a basic SA-aware planner looks like, with each targeting a different SA level. Traditional LLM agents often struggle with autonomous decision-making because they rely on external, manually designed performance metric that may be incorrect or not even be available in real time, real-world scenarios [39]. To address this limitation, researchers created the RaDAgent (Rational Decision-Making Agent) framework, which allows agents to independently learn the value of their actions from their own previous experiences [39]. The framework has the Experience Exploration stage, where the agent explores potential decisions, and the Utility Learning stage, where the agent uses an Elo-based mechanism to estimate the best decision [39]. These core mechanics show that internalized utility judgment is L2 SA of goal-relevance, and the results show that LLMs have a ~ 7.8% average improvement across four benchmarks (Game of 24, WebShop, ToolBench, RestBench) [39]. Furthermore, we have seen how L2/L3 understanding is shown by plan-ahead decomposition, and memory-efficient execution, which had 32% and 29% gains on synchronous and asynchronous Robotouille [42]. From this, we can conclude that internal utility scoring, and recursive re-injection are essential for SA-aware planning.

8.3 Adaptive and Population-Level SA

Beyond simple planning, SA can be applied to broader scopes of system maintenance and workflows. Within a single workflow, DenoiseFlow works via Noisy MDP uncertainty management [36], which we have previously explored. But across different task types, A²Flow uses a three-stage pipeline to extract operators (from case-specific to clustered to generalized) [43], which maps to the SA hierarchy. In this case, the framework's operator memory works as an episodic storage of SA by keeping historical outputs to improve context and decision-making [43]. The results show gains of 2.4% on general and 19.3% on embodied benchmarks over the baselines and even including a 37% reduction in resource usage [43]. At population scale, EvoFlow uses an evolutionary search method to keep a collection of SA strategies and outperforms on all seven benchmarks, with improvements ranging from 1.23% to 29.86% [41]. Given this evidence, we can see that LLM similarities within a workflow is a key to system SA. The results are summarized in Table 6.

Table 6. Agentic workflow architectures compared on SA-relevant dimensions. [21, 36, 41, 42, 43]

Architecture	Context Strategy	Performance Gain	Primary SA Failure Mitigated
ReCAP	Parent-plan re-injection	+32% synchronous and +29% asynchronous in Robotouille	Plan drift and L2 coherence loss
DenoiseFlow	Noisy MDP	33% to 44% Stage 3 recovery	Accumulated semantic ambiguity
A²Flow	Three-stage operator abstraction	+2.4% of general benchmarks and +19.3% embodied benchmarks	Static operator rigidity
EvoFlow	Niching evolutionary search	+1.23% to +29.86% on seven benchmarks	Single-strategy convergence
MemGPT	OS-style hierarchical memory	~100% vs ~0% at 3-hop KV	Context-window saturation

9. Closing

9.1 Conclusion

The two main sections of this literature review result in an answer to the initial research questions. The answer to the measurement question is a four-method setup where each method covers an SA level that the others miss. P(IK)/P(True) [15] is for L2 self-knowledge, SAD [18] is a multi-task behavioral benchmark for all three levels, linear probing [12] is for the detection of encoded L1 representations, and ToM tasks [16, 34] is for multi-agent L2 SA.

The answer to the failure mode question is a six-part model that is directly related to Endsley's levels. Hallucination [13] results in L1 and L2 failures with cascading effects like the propagation seen in clinical incidents [30, 32]. ToM fragility [34] results in fragile multi-agent L2 SA. Deceptive alignment [4, 6, 14, 20] results in adversarial L2 SA. Context saturation [21, 36, 37, 42] results in L3 collapse. Adversarial exploitation [17] creates the differences between local and global SA. And human OOTL [1, 3, 10, 19] results in SA loss at the supervision layer.

9.2 Research Gaps and Literature Review Limitations

There are multiple research gaps that were identified as a result of this literature review. First, no multi-agent SA measurement standard exists for LLM systems. Second, the CoT faithfulness gap [35] remains unresolved. Third, literature SA measurement in adversarial and dynamic situations is very limited.

There are a couple limitations to this review. The first is that the literature studies emphasize Endsley's model, and alternative frameworks weren't explored that much, despite their relevance to multi-agent settings. Next is that the LLM-specific literature is mainly from the years 2022 to 2026, and the rapid growth of LLM capabilities may quickly change the results of the findings of these papers.

References

[1] Bainbridge, L. (1983). Ironies of automation. *Automatica*, *19*(6), 775–779. https://doi.org/10.1016/0005-1098(83)90046-8

[2] Bandara, E., Gore, R., Shetty, S., Rajapakse, S., Kularathna, I., Karunarathna, P., Mukkamala, R., Foytik, P., Bouk, S. H., Rahman, A., Liang, X., Hass, A., Hewa, T., Keong, N. W., Zoysa, K. D., Withanage, A., & Loganathan, N. (2026). *A Practical Guide to Agentic AI Transition in Organizations* (arXiv:2602.10122). arXiv. https://doi.org/10.48550/arXiv.2602.10122

[3] Bansal, G., Wu, T., Zhou, J., Fok, R., Nushi, B., Kamar, E., Ribeiro, M. T., & Weld, D. S. (2021). *Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance* (arXiv:2006.14779). arXiv. https://doi.org/10.48550/arXiv.2006.14779

[4] Betley, J., Bao, X., Soto, M., Sztyber-Betley, A., Chua, J., & Evans, O. (2025). *Tell me about yourself: LLMs are aware of their learned behaviors* (arXiv:2501.11120). arXiv. https://doi.org/10.48550/arXiv.2501.11120

[5] Carichon, F., Khandelwal, A., Fauchard, M., & Farnadi, G. (2025). *The Coming Crisis of Multi-Agent Misalignment: AI Alignment Must Be a Dynamic and Social Process* (Version 2). arXiv. https://doi.org/10.48550/ARXIV.2506.01080

[6] Chen, S., Ma, S., Yu, S., Zhang, H., Zhao, S., & Lu, C. (2025). *Exploring Consciousness in LLMs: A Systematic Survey of Theories, Implementations, and Frontier Risks* (arXiv:2505.19806). arXiv. https://doi.org/10.48550/arXiv.2505.19806

[7] Dafoe, A., Hughes, E., Bachrach, Y., Collins, T., McKee, K. R., Leibo, J. Z., Larson, K., & Graepel, T. (2020). *Open Problems in Cooperative AI* (arXiv:2012.08630). arXiv. https://doi.org/10.48550/arXiv.2012.08630

[8] Du, S., Zhao, J., Shi, J., Xie, Z., Jiang, X., Bai, Y., & He, L. (2026). A Survey on the Optimization of Large Language Model-based Agents. *ACM Computing Surveys*, *58*(9), 1–37. https://doi.org/10.1145/3789261

[9] Endsley, M. R. (1995). Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, *37*(1), 32–64. https://doi.org/10.1518/001872095779049543

[10] Endsley, M. R., & Kiris, E. O. (1995). The Out-of-the-Loop Performance Problem and Level of Control in Automation. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, *37*(2), 381–394. https://doi.org/10.1518/001872095779064555

[11] Gorman, J. C., Cooke, N. J., & Winner, J. L. (2006). Measuring team situation awareness in decentralized command and control environments. *Ergonomics*, *49*(12–13), 1312–1325. https://doi.org/10.1080/00140130600612788

[12] Gurnee, W., & Tegmark, M. (2024). *Language Models Represent Space and Time* (arXiv:2310.02207). arXiv. https://doi.org/10.48550/arXiv.2310.02207

[13] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A Survey on Hallucination in Large Language Models: Principles, Taxonomy,

Challenges, and Open Questions. *ACM Transactions on Information Systems*, *43*(2), 1–55. https://doi.org/10.1145/3703155

[14] Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., Jermyn, A., Askell, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., … Perez, E. (2024). *Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training* (arXiv:2401.05566). arXiv. https://doi.org/10.48550/arXiv.2401.05566

[15] Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., … Kaplan, J. (2022). *Language Models (Mostly) Know What They Know* (arXiv:2207.05221). arXiv. https://doi.org/10.48550/arXiv.2207.05221

[16] Kosinski, M. (2023). *Evaluating Large Language Models in Theory of Mind Tasks*. https://doi.org/10.48550/ARXIV.2302.02083

[17] Kulkarni, N., Wu, X., Varia, S., & Bespalov, D. (2025). Agent vs. Agent: Automated Data Generation and Red-Teaming for Custom Agentic Workflows. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 912–936. https://doi.org/10.18653/v1/2025.emnlp-industry.62

[18] Laine, R., Chughtai, B., Betley, J., Hariharan, K., Scheurer, J., Balesni, M., Hobbhahn, M., Meinke, A., & Evans, O. (2024). *Me, Myself, and AI: The Situational Awareness Dataset (SAD) for LLMs* (arXiv:2407.04694). arXiv. https://doi.org/10.48550/arXiv.2407.04694

[19] Merat, N., Jamson, A. H., Lai, F. C. H., Daly, M., & Carsten, O. M. J. (2014). Transition to manual: Driver behaviour when resuming control from a highly automated vehicle. *Transportation Research Part F: Traffic Psychology and Behaviour*, *27*, 274–282. https://doi.org/10.1016/j.trf.2014.09.005

[20] Needham, J., Edkins, G., Pimpale, G., Bartsch, H., & Hobbhahn, M. (2025). *Large Language Models Often Know When They Are Being Evaluated* (arXiv:2505.23836). arXiv. https://doi.org/10.48550/arXiv.2505.23836

[21] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2024). *MemGPT: Towards LLMs as Operating Systems* (arXiv:2310.08560). arXiv. https://doi.org/10.48550/arXiv.2310.08560

[22] Panait, L., & Luke, S. (2005). Cooperative Multi-Agent Learning: The State of the Art. *Autonomous Agents and Multi-Agent Systems*, *11*(3), 387–434. https://doi.org/10.1007/s10458-005-2631-2

[23] Park, I. R., Kim, S., & Moon, J. (2025). Why do people resist AI-based autonomous cars?: Analyzing the impact of the risk perception paradigm and conditional value on public acceptance of autonomous vehicles. *PLOS ONE*, *20*(2), e0313143. https://doi.org/10.1371/journal.pone.0313143

[24] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). *Generative Agents: Interactive Simulacra of Human Behavior* (arXiv:2304.03442). arXiv. https://doi.org/10.48550/arXiv.2304.03442

[25] Salas, E., Prince, C., Baker, D. P., & Shrestha, L. (1995). Situation Awareness in Team Performance: Implications for Measurement and Training. *Human Factors: The Journal of the Human Factors and Ergonomics Society*, *37*(1), 123–136. https://doi.org/10.1518/001872095779049525

[26] Salmon, P., Stanton, N., Walker, G., & Green, D. (2006). Situation awareness measurement: A review of applicability for C4i environments. *Applied Ergonomics*, *37*(2), 225–238. https://doi.org/10.1016/j.apergo.2005.02.001

[27] Sanneman, L., & Shah, J. A. (2020). A Situation Awareness-Based Framework for Design and Evaluation of Explainable AI. In D. Calvaresi, A. Najjar, M. Winikoff, & K. Främling (Eds.), *Explainable, Transparent Autonomous Agents and Multi-Agent Systems* (Vol. 12175, pp. 94–110). Springer International Publishing. https://doi.org/10.1007/978-3-030-51924-7_6

[28] Sanneman, L., & Shah, J. A. (2022). The Situation Awareness Framework for Explainable AI (SAFE-AI) and Human Factors Considerations for XAI Systems. *International Journal of Human–Computer Interaction*, *38*(18–20), 1772–1788. https://doi.org/10.1080/10447318.2022.2081282

[29] Sarter, N. B., & Woods, D. D. (1991). Situation Awareness: A Critical But Ill-Defined Phenomenon. *The International Journal of Aviation Psychology*, *1*(1), 45–57. https://doi.org/10.1207/s15327108ijap0101_4

[30] Schulz, C. M., Krautheim, V., Hackemann, A., Kreuzer, M., Kochs, E. F., & Wagner, K. J. (2015). Situation awareness errors in anesthesia and critical care in 200 cases of a critical incident reporting system. *BMC Anesthesiology*, *16*(1), 4. https://doi.org/10.1186/s12871-016-0172-7

[31] Shin, W., Souza, R., Rosendo, D., Suter, F., Wang, F., Balaprakash, P., & Silva, R. F. da. (2025). *The (R)evolution of Scientific Workflows in the Agentic AI Era: Towards Autonomous Science* (arXiv:2509.09915). arXiv. https://doi.org/10.48550/arXiv.2509.09915

[32] Singh, H. (2006). Understanding diagnostic errors in medicine: A lesson from aviation. *Quality and Safety in Health Care*, *15*(3), 159–164. https://doi.org/10.1136/qshc.2005.016444

[33] Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2024). *Cognitive Architectures for Language Agents* (arXiv:2309.02427). arXiv. https://doi.org/10.48550/arXiv.2309.02427

[34] Ullman, T. (2023). *Large Language Models Fail on Trivial Alterations to Theory-of-Mind Tasks* (arXiv:2302.08399). arXiv. https://doi.org/10.48550/arXiv.2302.08399

[35] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2023). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* (arXiv:2201.11903). arXiv. https://doi.org/10.48550/arXiv.2201.11903

[36] Yan, Y., Peng, J., Li, S., Li, C., Shang, Y., Deng, C., Dai, R., Zhao, Y., Zhu, J., & Huang, Y. (2026). *DenoiseFlow: Uncertainty-Aware Denoising for Reliable LLM Agentic Workflows* (arXiv:2603.00532). arXiv. https://doi.org/10.48550/arXiv.2603.00532

[37] Yang, C., Srebro, N., & Li, Z. (2026). *Recursive Models for Long-Horizon Reasoning* (arXiv:2603.02112). arXiv. https://doi.org/10.48550/arXiv.2603.02112

[38] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models* (arXiv:2210.03629). arXiv. https://doi.org/10.48550/arXiv.2210.03629

[39] Ye, Y., Cong, X., Tian, S., Qin, Y., Liu, C., Lin, Y., Liu, Z., & Sun, M. (2023). *Rational Decision-Making Agent with Internalized Utility Judgment* (arXiv:2308.12519). arXiv. https://doi.org/10.48550/arXiv.2308.12519

[40] You, F., Yan, X., Zhang, J., & Cui, W. (2022). Design Factors of Shared Situation Awareness Interface in Human–Machine Co-Driving. *Information*, *13*(9), 437. https://doi.org/10.3390/info13090437

[41] Zhang, G., Chen, K., Wan, G., Chang, H., Cheng, H., Wang, K., Hu, S., & Bai, L. (2025). *EvoFlow: Evolving Diverse Agentic Workflows On The Fly* (arXiv:2502.07373). arXiv. https://doi.org/10.48550/arXiv.2502.07373

[42] Zhang, Z., Chen, T., Xu, W., Pentland, A., & Pei, J. (2025). *ReCAP: Recursive Context-Aware Reasoning and Planning for Large Language Model Agents* (arXiv:2510.23822). arXiv. https://doi.org/10.48550/arXiv.2510.23822

[43] Zhao, M., Wei, X., Shao, Y., Zhou, K., Yang, L., Rao, S., Zhan, J., & Chen, Z. (2025). *A²Flow: Automating Agentic Workflow Generation via Self-Adaptive Abstraction Operators* (arXiv:2511.20693). arXiv. https://doi.org/10.48550/arXiv.2511.20693